

Make a Static Webpage

CS 407 Intro To Linux

November 4, 2025

1 Introduction

In this lab you will write your first website. The website will be made with:

1. HTML - Language for the website content.
2. CSS - Cascading Style Sheets. Used for styling the website.
3. Javascript - Language used for adding some interactivity to the website.
4. Apache2 - The webserver software that will handle the details of HTTP/HTTPS for us.
5. Debian 13 - The operating system we'll run on
6. vim - The text editor you'll use to write the code.

In just a few minutes you'll have a real website on the internet! You'll be able to send a link to your website to friends and family to show off your work.

2 Setup

First you need to set up your machine. ssh into a Debian 13 droplet and run the following commands:

```
1 root@machine$ apt install vim apache2
```

When you install apache2, it will start running a default web page. Go to the ip address of your droplet in your browser on your phone and pc. You'll see the default page!

TODO add picture here

Now let's change the webpage really quick to make sure things are working. The webpage being shown to you, by default, is in `/var/www/html`. Let's change it and restart apache.

```
1 root@machine$ cd /var/www/html
2 root@machine$ ls
3 # you should see index.html
4 root@machine$ rm index.html
5 root@machine$ vim index.html
6 # create a new file. Make the file say "I am learning Linux"
7 root@machine$ cat index.html
```

```
8 | I am learning linux.  
9 | root@machine$ service apache2 restart
```

Now go to your droplet ip address in your browser. You should now see "I am learning Linux" and NOT the default page. Congratulations!

Note! Sometimes the browser will cache old versions of the webpage. If you are still seeing the default apache page, try the following:

1. Make absolutely sure that you followed the instructions above!! Are you sure you did everything?
2. You can force clear the cache (google for how, I think its something like CTRL + F5).
3. restart your browser
4. try a private browser window
5. try accessing the webpage on another device.

3 Your first HTML

HTML is *Hyper Text Markup Language*. Its the language you use for specifying the content like words, images, etc. that are shown on your website.

3.1 HTML Summary

HTML has a handful of useful tags for specifying the content of your page. For example you have:

1. h1 - header 1
2. h2 - header 2. A little smaller than header 1.
3. p - a paragraph
4. html - the tags that goes around the whole html
5. head - header information
6. body - the body of the webpage. The content the user will see.
7. ol - a numbered list of items
8. li - an item in the numbered list

Note that some tags come in pairs, with an open and close tag, like `<p></p>`, whereas other tags like the `<meta charset="UTF-8">` do not have a closing tag. If you want to read more about which tags are void and which are not, or you want to know more about HTML, you can search around the internet. Today we will only be doing some basic things so you can practice vim and you can see that your server works.

3.2 index.html

Let's replace index.html with some proper html instead of that text we put before. Open `/var/www/html/index.html` and put the following text:

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4     <meta charset="UTF-8">
5     <title>Linux Class</title>
6 </head>
7 <body>
8     <h1>About Me!</h1>
9     <p>My name is Fulano and I'm learning Linux.</p>
10    <h2>What I know</h2>
11    <p>I know alot about Linux</p>
12 </body>
13 </html>
```

Then restart apache2 and reload your page in the browser. Do you see the new webpage?

```
1 root@machine$ service apache2 restart
```

TODO add picture here

4 A Website With 2 Pages

Modify **index.html** so that it has a link to another page. The other page is called **linuxdetails.html** See below for the file contents. Make sure you save both files in **/var/www/html**.

4.1 index.html

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4     <meta charset="UTF-8">
5     <title>Linux Class</title>
6 </head>
7 <body>
8     <h1>About Me!</h1>
9     <p>My name is Fulano and I'm learning Linux.</p>
10    <h2>I know linux</h2>
11    <p><a href="linuxdetails.html">Click here for more information</a></p>
12 </body>
13 </html>
```

4.2 linuxdetails.html

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4     <meta charset="UTF-8">
5     <title>Linux Details</title>
6 </head>
7 <body>
8     <h1>What I know</h1>
9     <p>Here are some things I know about Linux.</p>
10    <h2>mkdir</h2>
11    <p>Create a directory</p>
12    <h2>cp</h2>
13    <p>Copy a file</p>
14    <h2>mv</h2>
15    <p>Rename a file</p>
16    <h2>rm</h2>
17    <p>Delete a file</p>
18 </body>
19 </html>
```

4.3 Make it Better!

At this point you know alot about Linux. Expand your **linuxdetails.html** page so it says 10 things you know. For inspiration, consider the following:

- cp
- cd
- touch
- ssh
- mkdir
- grep
- echo
- sftp
- mv
- pipes
- vim
- rm
- awk
- bash

4.4 Can you Add a Third Page?

Why not add another link to index.html like this:

```

1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <title>Linux Class</title>
6 </head>
7 <body>
8   <h1>About Me!</h1>
9   <p>My name is Fulano and I'm learning Linux.</p>
10  <h2>I know linux</h2>
11  <p><a href="linuxdetails.html">Click here for more information</a></p>
12  <h2>I also know how to cook</h2>
13  <p><a href="cooking.html">Click here for my recipes</a></p>
14 </body>
15 </html>

```

and then add another file **cooking.html**. I'm going to give you an example cooking.html below, but you should create your own.

```

1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <title>Recipes</title>
6 </head>
7 <body>
8   <h1>My recipes</h1>
9
10  <h2>Grilled cheese</h2>
11  <ol>
12    <li>Put mayo on 2 slices of bread.</li>
13    <li>Put a few slices of meltable cheese like cheddar in the bread.</li>
14    <li>Melt some butter in a pan and toast the sandwich.</li>
15    <li>Remove when bread is golden brown and cheese is melted.</li>
16  </ol>
17
18 </body>
19 </html>

```

5 Adding CSS

CSS is *Cascading Style Sheets* and is the language used for specifying the style of your webpage. You can do quite a bit with CSS! For now we'll just use it to change the page colors.

Add a file called **style.css** to `/var/www/html` and add a link to it in the headers of your html pages. Below you can find an example of how **index.html** has the stylesheet in the header.

5.1 style.css

```
1 body {
2     background-color: #F5F5F5;
3     color: #0A0A11;
4 }
5
6 h1 {
7     color: #0108A3;
8 }
9
10 h2 {
11     color: #0008EE;
12 }
```

5.2 index.html

Note how in the page head you now see a link to style.css

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4     <meta charset="UTF-8">
5     <link rel="stylesheet" href="style.css">
6     <title>Linux Class</title>
7 </head>
8 <body>
9     <h1>About Me!</h1>
10    <p>My name is Fulano and I'm learning Linux.</p>
11    <h2>I know linux</h2>
12    <p><a href="linuxdetails.html">Click here for more information</a></p>
13    <h2>I also know how to cook</h2>
14    <p><a href="cooking.html">Click here for my recipes</a></p>
15 </body>
16 </html>
```

6 Javascript

And now we will add Javascript. Javascript is a language that allows you to add some functionality to the webpage. Some people like to write websites that don't use javascript, but perform all functionality on the server aka the "backend". Other people like to write javascript-heavy websites that do most of their work in the "frontend". In this example you'll get a peak at what javascript looks like.

Add a file called **counter.js** to **/var/www/html** and then modify **index.html** so it incorporates the javascript.

6.1 counter.js

```
1 let clickCount = 0;
2
3 /*
4  * This function gets the 'clickDisplay' element from your HTML
5  * and changes the text. Have a look in index.html and you will see
6  * a paragraph with id "clickDisplay".
7  */
8 function countClicks() {
9     clickCount++;
10    document.getElementById('clickDisplay').innerText =
11        `Button clicked ${clickCount} time${clickCount === 1 ? '' : 's'}`;
12 }
```

Be careful with the quotes in counter.js. Before the word "Button" and before the ";" there is a backtick, like ```, and not a quote like `'`. The backticks are used in Javascript for string interpolation.

6.2 index.html

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4     <meta charset="UTF-8">
5     <link rel="stylesheet" href="style.css">
6     <title>Linux Class</title>
7 </head>
8 <body>
9     <h1>About Me!</h1>
10    <p>My name is Fulano and I'm learning Linux.</p>
11
12    <h2>I know Linux</h2>
13    <p><a href="linuxdetails.html">Click here for more information</a></p>
14
15    <h2>I also know how to cook</h2>
16    <p><a href="cooking.html">Click here for my recipes</a></p>
17
18    <h2>Click Counter</h2>
19    <button onclick="countClicks()">Click me!</button>
20    <p id="clickDisplay">Button clicked 0 times</p>
21    <script src="counter.js" defer></script>
```

```
22  
23 </body>  
24 </html>
```

7 Congratulations!

You reached the end! You were able to deploy a real static website on Debian Linux with the Apache2 webserver! Pat yourself on the back and feel proud! But know, this is just the tip of the iceberg. There's still a ton to learn.

While you wait for the next lesson, you should look into more about html. There are many more tags besides h1, h2, p, and a. And CSS can do many more things than you've seen.